

Social Media Platform

Complete Project Plan & Implementation Guide

[Next.js](#)[Tailwind CSS](#)[Supabase](#)[Cloudinary](#)

Phase 1: Project Setup & Initialization

- **Frontend Setup:** Initialize your Next.js application using App Router: `npx create-next-app@latest my-social-app`.
- **Styling & Icons:** Ensure Tailwind CSS is configured. Install `lucide-react` for beautiful, consistent UI icons.
- **Supabase Integration:** Create a new project on Supabase. Copy the API URL and anon key into your Next.js `.env.local` file. Install the SDK: `npm install @supabase/supabase-js`.
- **Cloudinary Setup:** Create a free Cloudinary account. Configure an "Unsigned Upload Preset" to allow users to upload profile pictures and post images directly from the frontend.

Phase 2: Database Architecture (PostgreSQL)

Run this schema in your Supabase SQL Editor to establish relationships:

```

-- 1. Profiles Table
CREATE TABLE profiles (
  id UUID REFERENCES auth.users(id) ON DELETE CASCADE PRIMARY KEY,
  username TEXT UNIQUE NOT NULL,
  full_name TEXT, avatar_url TEXT, bio TEXT,
  created_at TIMESTAMP WITH TIME ZONE DEFAULT timezone('utc'::text,
now())
);

-- 2. Posts Table
CREATE TABLE posts (
  id UUID DEFAULT gen_random_uuid() PRIMARY KEY,
  user_id UUID REFERENCES profiles(id) ON DELETE CASCADE NOT NULL,
  content TEXT, image_url TEXT,
  created_at TIMESTAMP WITH TIME ZONE DEFAULT timezone('utc'::text,
now())
);

-- 3. Friendships Table
CREATE TABLE friendships (
  id UUID DEFAULT gen_random_uuid() PRIMARY KEY,
  requester_id UUID REFERENCES profiles(id) ON DELETE CASCADE,
  receiver_id UUID REFERENCES profiles(id) ON DELETE CASCADE,
  status TEXT CHECK (status IN ('pending', 'accepted', 'rejected'))
  DEFAULT 'pending'
);

```

Important: Set up tables for comments, likes, and messages following the same pattern, ensuring ON DELETE CASCADE is used.

Phase 3: Authentication & Triggers

- **User Auth:** Implement Sign Up and Login components using Supabase's built-in authentication (Email/Password or Google OAuth).
- **Automated Profiles:** Write a PostgreSQL trigger in Supabase (AFTER INSERT ON auth.users) to automatically generate a row in your profiles table whenever a new user registers.

- **Profile Management:** Build the `/profile/[username]` page allowing users to update their bio and upload avatars via Cloudinary.

Phase 4: Core Features (Feed & Posts)

- **Post Creation:** Build a UI component with a text area and image upload button. Route image uploads to Cloudinary first, then save the returned URL + text to the Supabase posts table.
- **News Feed:** Fetch posts dynamically for the Home page. Use Supabase relational queries to join the posts table with the profiles table so you can display the author's avatar and name.
- **Interactions:** Implement "Like" buttons and a commenting system attached to specific Post IDs.

Phase 5: Friend System

- **Sending Requests:** Add an "Add Friend" button on user profiles that inserts a row into the friendships table with a pending status.
- **Managing Requests:** Create a dedicated notification/requests page where users can accept or reject pending requests (updating the status accordingly).

Phase 6: Real-time Messaging & Chat

- **Chat UI:** Design a direct messaging layout similar to modern social apps (contacts on the left, chat window on the right).
- **Supabase Realtime:** Utilize Supabase's `channel()` API to subscribe to INSERT events on the messages table.

- **Live Updates:** Ensure that when a new message is sent, the UI updates instantly for the receiver without requiring a page refresh.

Phase 7: Polish & Deployment

- **UI Refinement:** Audit your app to ensure fully responsive design across mobile and desktop views using Tailwind utility classes. Add loading skeletons for better UX.
- **Security:** Enable Row Level Security (RLS) in Supabase to prevent unauthorized data manipulation.
- **Vercel Deployment:** Push code to GitHub. Import the repository into Vercel and securely add your Supabase and Cloudinary credentials to Vercel's Environment Variables.